4 Week 4: Formal & Informal Technical Reports

4.1 Documenting the Methodology Chapter: Research Design, System Modeling, and Evaluation Protocols

The methodology chapter of an IT research project or graduation thesis sets the logical parameters for how the proposed computing solution is designed, constructed, and scientifically validated.

Similar to how the introduction chapter funnels general context into a specific problem using the Inverted Pyramid Model, the methodology chapter functions as the rigorous conceptual engine that bridges the research gap through systematic artifact creation and empirical verification.

- **The Design Science Framework as a Metaphor:** In computing and technical disciplines, the methodology chapter acts as the abstract architecture and structural blueprint of your system. Rather than a casual chronological log of what was coded, it formally maps the logical constraints, data patterns, architectural parameters, and validation environments that govern the proposed solution.

- **Applying the Model in IT Methodology:** For an IT thesis, the methodology typically begins by identifying the overarching development framework—such as **Design Science Research (DSR)** or **Action Research**—to justify the stages of problem-solving. This creates an academic foundation that moves the study from a basic software development project into a piece of documented, peer-reviewed engineering research.

- **Abstracting from Code to Engineering:** According to Swales and Feak (2012), academic transparency mandates that the methods chapter provides sufficient explicit detail to enable peer researchers to replicate your work precisely. In IT, this translates from informal code narratives into formal systems engineering documentation, emphasizing clarity, structural precision, and scientific reproducibility.

- **The Path to Validation:** Finally, the methodology chapter culminates in a comprehensive evaluation protocol. It defines the exact technical performance metrics, algorithmic criteria, or human-centric validation methods used to prove that the computing solution effectively addresses the research gap identified in Week 3 (Diyana Binti Maznun et al., 2017; Veerappan et al., 2024).

4.2 Core Methodology Artifacts and Specifications

To ensure absolute document integrity and scientific validity, undergraduate researchers must formally document **five critical structural components** within their methodology chapters:

1. **Research Design Framework:** Students must explicitly define the overarching scientific approach guiding their technical development. For IT and software engineering projects, this typically follows the Design Science Research Process (DSRP). This section justifies the structured steps taken to move from an identified real-world problem to a designed, functional artifact.

2. **System Architecture & Logical Modeling:** Rather than discussing raw source code, the researcher must document the abstract logical flow of the system. This requires the inclusion of formal structural models, such as Unified Modeling Language (UML) Diagrams (e.g., Use Case, Class, or Activity diagrams), Data Flow Diagrams (DFD) mapping system inputs/outputs, or System Block Diagrams illustrating structural tiers.

3. **Data Engineering & Processing Methodology:** Document exactly how data enters, moves through, and exits the system. Students must clearly specify data sourcing (e.g., public APIs, static datasets, simulated network traffic), data preprocessing steps (e.g., filtering, cleaning, or normalization), and the core algorithms or mathematical formulas used to transform inputs into outputs.

Copyright © 2026 Dalya Raad AlAni

4. **Technical Environment & Development Paradigms:** Document the foundational technological ecosystem required to make the project reproducible for another developer. Students should focus on high-level environment parameters rather than fluctuating minor patch levels: primary programming languages, core web/mobile frameworks, architectural styles (e.g., RESTful APIs, Microservices), and the target operating environment.

5. **Evaluation, Testing & Validation Protocols:** A methodology is incomplete without a clear plan for how the system's success will be scientifically measured. Students must establish clear evaluation metrics suited to their specific project type: technical performance metrics (e.g., latency, throughput), system accuracy metrics (e.g., F1-Score, Mean Squared Error), or User Acceptance Testing (UAT) using standardized metrics like the System Usability Scale (SUS).

IT Example: Distributed IoT Edge Anomaly Detection System

1. **Research Design Framework:** The project adopts the Design Science Research (DSR) paradigm, executing an iterative process of artifact design, edge deployment, and empirical evaluation to address vulnerabilities in resource-constrained networks.

2. **System Architecture & Logical Modeling:** A high-level system block diagram partitions the solution into three distinct structural tiers: the IoT Edge Node sensing layer, the Local Gateway processing hub, and the centralized Cloud Analytics dashboard. An accompanying UML sequence diagram maps data transmission protocols between tiers.

3. **Data Engineering & Processing Methodology:** Environmental telemetry data is scraped from a public MQTT broker stream. The raw data undergoes min-max normalization to fit a 0-1 scale before being routed through a localized Lightweight Isolation Forest algorithm to isolate anomalous traffic vectors.

4. **Technical Environment & Development Paradigms:** The system is compiled using Python within a containerized Linux environment. The microservices communicate asynchronously via lightweight REST APIs to optimize bandwidth consumption on local gateway machines.

5. **Evaluation & Testing Protocols:** The model's efficacy is evaluated using two distinct criteria: technical performance (measuring processing latency and CPU utilization on the edge node under simulated load constraints) and algorithmic accuracy (calculating precision, recall, and F1-scores against a pre-labeled network intrusion dataset).

4.3 Sub-Themes & Application: Architectural Modeling, Data Processing Logic, Evaluation Matrices

Contextual Framing in Methodology means positioning the selected system design and development steps within established software engineering patterns. For IT bachelor projects, this involves detailing the architectural paradigms, logic rules, and validation frameworks that relate directly to your problem domain. This framing ensures your audience understands why specific technical selections were made over alternative tools or design architectures.

Technical Research Baseline Linkage: The methodology must be explicitly justified based on prior work presented in the baseline. If the Week 3 baseline showed that current systems fail due to high processing overhead, the Week 4 methodology must explicitly illustrate how the proposed system architecture minimizes memory consumption (Alqahtani et al., 2024).

Copyright © 2026 Dalya Raad AlAni

Concept	Application
Research Design Framework	Explicitly mapping an IT project to the Design Science Research Process (DSRP) steps instead of just writing a chronological diary of the software development lifecycle.
System Architecture Modeling	Using formal UML Component diagrams to illustrate how a mobile frontend decouples from a serverless backend database layer, rather than using non-standard sketches.
Data Engineering Logic	Documenting the specific feature extraction steps and mathematical formula used for mathematical normalization of data before feeding it into a processing engine.
Technical Environment Framing	Detailing the high-level ecosystem requirements (e.g., Cross-Platform Mobile Environment, Relational Database Tier) rather than volatile software update sub-versions.
Evaluation Protocols	Defining the rigorous mathematical formulas for calculating Precision, Recall, and F1-scores to evaluate an image processing algorithm's accuracy.

Table 1: Additional Examples to Illustrate Key Methodology Concepts

4.4 A Complete Methodology Example

Project: "Improving Network Security in Small Businesses Using an Adaptive Firewall"

(**Research Design Framework**) This study utilizes the Design Science Research (DSR) framework to engineer, implement, and validate a computing artifact designed to mitigate dynamic network security vulnerabilities within resource-constrained corporate environments. The methodology progresses from requirements analysis to logical modeling, system prototyping, and empirical performance testing.
(**System Architecture & Logical Modeling**) The structural architecture of the proposed adaptive firewall system is modeled as a multi-tier proxy environment. The edge layer captures inbound TCP/IP packet strings and pipes them to an intermediate filtering pipeline. To maintain absolute structural transparency, the logical execution path of the filtering engine is mapped using a formal UML Activity Diagram detailing packet evaluation loops.
(**Data Engineering & Processing Methodology**) The system processes active network telemetry strings. Incoming raw packet objects undergo structured header extraction to isolate three core features: source IP address, destination port configuration, and packet size payload bytes. Data instances containing missing or corrupted header flags are discarded immediately at the ingestion boundary. Valid features are subsequently normalized through an algorithmic hashing function to compute real-time anomaly scores.
(**Technical Environment & Development Paradigms**) The computing artifact is designed using an Object-Oriented Programming (OOP) paradigm and compiled within a native Linux ecosystem. The backend processing module utilizes Python infrastructure due to its robust native socket programming libraries. All system metrics and filtering rule states are managed inside a structured relational database layer to ensure lightweight data persistence.
(**Evaluation & Testing Protocols**) The system artifact is evaluated across two primary verification vectors:

1. *Technical Throughput Performance:* The firewall instance is subjected to a simulated network load test using traffic generator utilities to measure processing latency overhead under variable stress bounds (ranging from 1,000 to 10,000 requests per second).

2. *Threat Mitigation Efficacy:* Algorithmic accuracy is verified by processing a standardized, pre-labeled cyber threat dataset. System performance is quantified by mapping true-positive and

Copyright © 2026 Dalya Raad AlAni

false-positive outcomes to compute definitive Precision and Recall metrics using the standard formulas:

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}} \tag{1}$$

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}} \tag{2}$$

4.5 The Grammar of Methodology: Split-Tense Protocol

In academic IT writing, the methodology chapter demands a precise linguistic division between permanent architectural rules (Present Simple) and the completed execution steps taken by the researcher (Past Simple).

Methodology Move	**Tense**	**Grammar Structure**	**Student-Facing Example**
Research Design	Present Simple	Subject + Verb 1 (s/es)	"Design Science Research **governs** the loop, and the system **utilizes** microservices."
System Visuals	Present Simple	Subject + Verb 1 (s/es)	"Figure 4.2 **illustrates** the data schema, and Equation 4.1 **defines** the threshold."
Data Processing	Past (Passive)	Subject + was/were + Verb 3	"Raw telemetry data **was extracted**, and anomalous records **were removed**."
Implementation	Past (Active)	Subject + Verb 2 (or Passive)	"The team **implemented** a model, and the code **was compiled** on a server."
Evaluation Execution	Past (Passive)	Subject + was/were + Verb 3	"System performance **was evaluated**, and network latency **was recorded**."

Table 2: Split-Tense Protocol in Methodology Chapter

Copyright © 2026 Dalya Raad AlAni

References

Alqahtani, M., Rabbani, M., and Schmidt, K. (2024). Standardized reporting guidelines for software engineering research projects. *Journal of Systems and Software*, 210:111–125.

Diyana Binti Maznun, M., Monsefi, R., and Nimehchisalem, V. (2017). Undergraduate ESL students' difficulties in writing the introduction and methodology for research reports. *Advances in Language and Literary Studies*, 8(1):9.

Swales, J. M. and Feak, C. B. (2012). *Academic writing for graduate students: Essential tasks and skills*. University of Michigan Press, Ann Arbor, 3rd edition.

Veerappan, V., Ahmad, M., Balakrishnan, K., Aris, M. A., and Suan, W. H. (2024). Genre analysis of structural sections in engineering undergraduate technical reports. *F1000Research*, 11:163.

Copyright © 2026 Dalya Raad AlAni

Dalya Raad Al-Ani

Digitally signed by Dalya Raad Al-Ani